

Rockchip Developer Guide Linux GMAC

ID: RK-KF-YF-130

Release Version: V1.0.0

Release Date: 2021-01-16

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides a usage document for the Ethernet GMAC interface on the Rockchip platform, intended to solve most Ethernet-related issues.

Product Version

Chipset	Kernel Version
ROCKCHIP Chip	3.10/4.4/4.19

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Wu Dachao	2021-01-16	Initial Version

Contents

Rockchip Developer Guide Linux GMAC

1. Code Location
2. DTS
3. Debugging PHY Register Read and Write
 - 3.1 Linux 3.10
 - 3.2 Other Versions
4. MAC Address
5. Loopback Testing
6. RGMII Delayline
7. LED Lights
8. WOL
9. MAC To MAC Direct Connection
10. Jumbo Frame
11. PTP1588
 - 11.1 PC Master and RK Slave
 - 11.2 RK Master and PC Slave
12. Problem Analysis
 - 12.1 DMA Initialization Failed
 - 12.2 PHY Initialization Failure
 - 12.3 Link Issues
 - 12.4 Data Inconsistency
 - 12.4.1 TX
 - 12.4.2 RX
 - 12.5 TX queue0 timeout

1. Code Location

The hardware-related driver code for the Ethernet module mainly includes GMAC and PHY. The PHY driver generally uses a universal PHY driver; if there is a need to modify special registers, please use the corresponding PHY driver, and the code is located in `drivers/net/phy`. Additionally, RK322x/RK3328 comes with a built-in 100M PHY chip.

- Linux 3.10 GMAC driver code `driver/net/ethernet/rockchip/gmac/*`
- Other kernel GMAC driver code for versions higher than 3.10, the GMAC driver code location is `drivers/net/ethernet/stmicro/stmmac/*`
- RK internal EPHY driver code `drivers/net/phy/rockchip.c`

2. DTS

DTS configuration refers to:

`Documentation/devicetree/bindings/net/rockchip-dwmac.txt`

```

gmac: ethernet@ff290000 {
    compatible = "rockchip,rk3288-gmac";
    reg = <0xff290000 0x10000>;
    interrupts = <GIC_SPI 27 IRQ_TYPE_LEVEL_HIGH>;
    interrupt-names = "macirq";
    rockchip,grf = <&grf>;
    clocks = <&cru SCLK_MAC>,
            <&cru SCLK_MAC_RX>, <&cru SCLK_MAC_TX>,
            <&cru SCLK_MACREF>, <&cru SCLK_MACREF_OUT>,
            <&cru ACLK_GMAC>, <&cru PCLK_GMAC>;
    clock-names = "stmmaceth",
                  "mac_clk_rx", "mac_clk_tx",
                  "clk_mac_ref", "clk_mac_refout",
                  "aclk_mac", "pclk_mac";
    phy-mode = "rgmii";
    pinctrl-names = "default";
    pinctrl-0 = <&rgmii_pins /*&rmii_pins*/>;

    clock_in_out = "input";
    snps,reset-gpio = <&gpio4 7 0>;
    snps,reset-active-low;
    snps,reset-delays-us = <0 10000 1000000>;

    assigned-clocks = <&cru SCLK_MAC>;
    assigned-clock-parents = <&ext_gmac>;
    tx_delay = <0x30>;
    rx_delay = <0x10>;

    status = "ok";
};

```

Board-level configuration requires attention to the following parts:

- phy-mode: Mainly divided into RMII and RGMII modes
- snps,reset-gpio: The hardware reset pin of PHY
- snps,reset-delays-us: The reset timing of PHY, the three times represent the reset timing of different stages of PHY, which are different for different PHYs. If the snps,reset-active-low attribute is used, it means the three times represent the time when the Reset pin is pulled high, pulled low, and then pulled high again; if the snps,reset-active-high attribute is used, it is the opposite
- phy-supply: If the power supply of PHY is a constant supply method, it does not need to be configured; otherwise, the corresponding regulator needs to be configured
- Clock configuration: Please refer to Chapter 3 of this document
- pinctrl: The configurations are different between RGMII and RMII modes, and for clock modes, if the pin is an output clock pin, the driving strength of this pin is generally different. For example, in RMII mode, when the ref_clock pin outputs the clock, the driving strength is also configured to be larger
- tx_delay/rx_delay: This attribute needs to be configured under RGMII mode, please refer to Chapter 8 of RGMII Delayline in this document

Due to the variety of configurations under different modes for different chips, please refer to another document [Rockchip_Developer_Guide_Linux_GMAC_Mode_Configuration_EN.pdf](#).

3. Debugging PHY Register Read and Write

The driver provides interfaces for reading and writing registers, and there are currently two sets of interfaces on different kernel versions.

Path: `/sys/bus/mdio_bus/devices/stmmac-0:00`, where `stmmac-0:00` indicates that the PHY address is 0.

3.1 Linux 3.10

```
/sys/bus/mdio_bus/devices/stmmac-0:00/phy_reg  
/sys/bus/mdio_bus/devices/stmmac-0:00/phy_regValue
```

- Write

For example, to write 0xabcd into Reg0

```
echo 0x00 > /sys/bus/mdio_bus/devices/stmmac-0:00/phy_reg  
echo 0xabcd > /sys/bus/mdio_bus/devices/stmmac-0:00/phy_regValue
```

- Read

For example, to read the value of Reg0

```
echo 0x00 > /sys/bus/mdio_bus/devices/stmmac-0:00/phy_reg  
cat /sys/bus/mdio_bus/devices/stmmac-0:00/phy_regValue
```

3.2 Other Versions

```
/sys/bus/mdio_bus/devices/stmmac-0:00/phy_registers
```

- Write

For example, to write 0xabcd to Reg0:

```
echo 0x00 0xabcd > /sys/bus/mdio_bus/devices/stmmac-0:00/phy_registers
```

- Read

```
cat /sys/bus/mdio_bus/devices/stmmac-0:00/phy_registers
```

This command will read all registers from 0 to 31, so you can view the corresponding register values.

4. MAC Address

The current strategy for reading MAC addresses prioritizes the MAC address within the DTB (which uboot also writes to), followed by the MAC address burned into the IDB. If this address complies with the specifications, it is used; if it does not comply or is not burned, a randomly generated address is used (the MAC address will change upon reboot). In versions of RK3399, RK3328/RK3228H, and later, the strategy has been refined: priority is given to the MAC address burned into the IDB or vendor Storage. If this address complies with the specifications, it is used; if it does not comply or is not burned, a randomly generated MAC address is saved to the Vendor partition and used, which will not be lost upon reboot or factory reset.

MAC address burning tool reference document: [Rockchip User Guide RKDevInfoWriteTool EN.pdf](#).

5. Loopback Testing

Loopback testing primarily involves MAC and PHY loopbacks, for specific details refer to the document "Rockchip_Developer_Guide_Linux_GMAC_RGMII_Delayline_EN.pdf", which provides explanations for the phy_lb and mac_lb nodes.

6. RGMII Delayline

RGMII interface provides tx and rx delaylines for adjusting RGMII timing. For obtaining the appropriate RGMII Delayline, please refer to the document

"Rockchip_Developer_Guide_Linux_GMAC_RGMII_Delayline_CN.pdf".

7. LED Lights

PHY has its own LED control. Below are the macphy configurations for RK3228 and RK3328. For other external PHYs, please refer to their respective datasheets. Below are the LED configurations for RK3228 and RK3328:

- RK3228: It is necessary to apply the patch `kernel_4.4_rk322x_phy_led_control.patch`.
- RK3328: By configuring the iomux in the dts, for example, controlling the led through rx and link, then configure the corresponding pinctrl.

```

phy: phy@0 {
    compatible = "ethernet-phy-id1234.d400", "ethernet-phy-ieee802.3-c22";
    reg = <0>;
    clocks = <&cru SCLK_MAC2PHY_OUT>;
    resets = <&cru SRST_MACPHY>;
    pinctrl-names = "default";
    pinctrl-0 = <&fephyled_rxm1 &fephyled_linkm1>;
    phy-is-integrated;
};

```

8. WOL

Wake On Lan functionality varies in configuration registers for each PHY. The patches currently included encompass RTL8211E/F and RTL8201F.

9. MAC To MAC Direct Connection

Refer to the document "Rockchip_Developer_Guide_Linux_MAC_TO_MAC_EN.pdf".

10. Jumbo Frame

Starting from the RV1126/1109 chip, Jumbo Frame 9K is available which requiring the test network MTU to be configured to 9000. Below are the test results:

```

<pre>[root@Puma:~]# ping -s 9000 192.168.1.100
PING 192.168.1.100 (192.168.1.100) 9000(9028) bytes of data.
9008 bytes from 192.168.1.100: icmp_seq=1 ttl=64 time=0.784 ms
9008 bytes from 192.168.1.100: icmp_seq=2 ttl=64 time=0.675 ms
9008 bytes from 192.168.1.100: icmp_seq=3 ttl=64 time=0.666 ms
9008 bytes from 192.168.1.100: icmp_seq=4 ttl=64 time=0.656 ms
9008 bytes from 192.168.1.100: icmp_seq=5 ttl=64 time=0.677 ms
9008 bytes from 192.168.1.100: icmp_seq=6 ttl=64 time=0.637 ms
9008 bytes from 192.168.1.100: icmp_seq=7 ttl=64 time=0.641 ms
9008 bytes from 192.168.1.100: icmp_seq=8 ttl=64 time=0.692 ms
9008 bytes from 192.168.1.100: icmp_seq=9 ttl=64 time=0.656 ms

```

11. PTP1588

Starting from the RV1126/1109 chip, PTP1588 is supported. Below are the test results:

11.1 PC Master and RK Slave

```
ubuntu@thinkpad:~$ sudo ptp4l -i enp0s31f6 -m -H
ptp4l[1790161.443]: selected /dev/ptp0 as PTP clock
ptp4l[1790161.443]: port 1: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[1790161.443]: port 0: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[1790168.489]: port 1: LISTENING to MASTER on ANNOUNCE_RECEIPT_TIMEOUT_EXPIRES
ptp4l[1790168.489]: selected local clock 54e1ad.ffff.dfa454 as best master
ptp4l[1790168.490]: assuming the grand master role
```

```
[root@Puma:~]# ptp4l -i eth0 -m -H -s
ptp4l[39.868]: selected /dev/ptp0 as PTP clock
[ 39.871092] rk_gmac-dwmac ffc40000.ethernet eth0: stmmac_hwtstamp_set config flags:0x0,
tx_type:0x1, rx_filter:0xc
[ 39.872029] stmmac_hwtstamp_set, value: 0x17e03
ptp4l[39.870]: port 1: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[39.871]: port 0: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[41.251]: port 1: new foreign master 54e1ad.ffff.dfa454-1
[ 43.817340] rk_gmac-dwmac ffc40000.ethernet eth0: stmmac_hwtstamp_set config flags:0x0,
tx_type:0x1, rx_filter:0xc
[ 43.818262] stmmac_hwtstamp_set, value: 0x17e03
ptp4l[45.251]: selected best master clock 54e1ad.ffff.dfa454
ptp4l[45.251]: port 1: LISTENING to UNCALIBRATED on RS_SLAVE
ptp4l[49.251]: master offset      -1608 s0 freq      +0 path delay      5691
ptp4l[50.251]: master offset      -5579 s0 freq      +0 path delay      9435
ptp4l[51.251]: master offset      -4831 s2 freq      +748 path delay      9435
ptp4l[51.251]: port 1: UNCALIBRATED to SLAVE on MASTER_CLOCK_SELECTED
ptp4l[52.251]: master offset      12189 s2 freq    +12937 path delay      7563
ptp4l[53.251]: master offset      14413 s2 freq    +18818 path delay      8287
ptp4l[54.251]: master offset      10712 s2 freq    +19441 path delay      8861
ptp4l[55.251]: master offset       7185 s2 freq    +19127 path delay      8861
ptp4l[56.251]: master offset       3234 s2 freq    +17332 path delay      9435
ptp4l[57.251]: master offset       1787 s2 freq    +16855 path delay      9454
ptp4l[58.251]: master offset        785 s2 freq    +16389 path delay      9454
ptp4l[59.251]: master offset         89 s2 freq    +15928 path delay      9473
ptp4l[60.251]: master offset         31 s2 freq    +15897 path delay      9454
ptp4l[61.251]: master offset        -71 s2 freq    +15804 path delay      9454
ptp4l[62.251]: master offset       -100 s2 freq    +15754 path delay      9406
ptp4l[63.251]: master offset        -27 s2 freq    +15797 path delay      9406
ptp4l[64.251]: master offset        -69 s2 freq    +15747 path delay      9395
ptp4l[65.251]: master offset         29 s2 freq    +15824 path delay      9395
ptp4l[66.251]: master offset       -73 s2 freq    +15731 path delay      9395
ptp4l[67.251]: master offset         32 s2 freq    +15814 path delay      9388
ptp4l[68.251]: master offset       -20 s2 freq    +15772 path delay      9388
ptp4l[69.251]: master offset      -104 s2 freq    +15682 path delay      9395
```

```
ptp4l[70.251]: master offset      -56 s2 freq +15699 path delay 9395
ptp4l[71.251]: master offset      24 s2 freq +15762 path delay 9388
ptp4l[72.251]: master offset      11 s2 freq +15756 path delay 9395
```

11.2 RK Master and PC Slave

```
[root@Puma:/]# ptp4l -i eth0 -m -H
ptp4l[15.668]: selected /dev/ptp0 as PTP clock
ptp4l[15.670]: port 1: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[15.670]: port 0: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[22.120]: port 1: LISTENING to MASTER on ANNOUNCE_RECEIPT_TIMEOUT_EXPIRES
ptp4l[22.120]: selected local clock aadc46.ffff.5da6d9 as best master
ptp4l[22.121]: assuming the grand master role
```

```
ubuntu@thinkpad:~$ sudo ptp4l -i enp0s31f6 -m -H -s
ptp4l[1879661.603]: selected /dev/ptp0 as PTP clock
ptp4l[1879661.603]: port 1: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[1879661.603]: port 0: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[1879662.249]: port 1: new foreign master aadc46.ffff.5da6d9-1
ptp4l[1879665.849]: selected best master clock aadc46.ffff.5da6d9
ptp4l[1879665.849]: port 1: LISTENING to UNCALIBRATED on RS_SLAVE
ptp4l[1879667.649]: master offset      49 s0 freq  -9515 path delay 9364
ptp4l[1879668.549]: master offset     128 s2 freq  -9436 path delay 9338
ptp4l[1879668.549]: port 1: UNCALIBRATED to SLAVE on MASTER_CLOCK_SELECTED
ptp4l[1879669.449]: master offset     256 s2 freq  -9180 path delay 9338
ptp4l[1879670.349]: master offset    -230 s2 freq  -9589 path delay 9338
ptp4l[1879671.249]: master offset    -399 s2 freq  -9827 path delay 9360
ptp4l[1879672.149]: master offset     142 s2 freq  -9406 path delay 9360
ptp4l[1879673.049]: master offset     232 s2 freq  -9273 path delay 9347
ptp4l[1879673.949]: master offset    -303 s2 freq  -9739 path delay 9347
ptp4l[1879674.849]: master offset    -267 s2 freq  -9794 path delay 9338
ptp4l[1879675.749]: master offset     327 s2 freq  -9280 path delay 9335
ptp4l[1879676.649]: master offset     405 s2 freq  -9104 path delay 9335
ptp4l[1879677.549]: master offset    -156 s2 freq  -9543 path delay 9335
ptp4l[1879678.449]: master offset    -178 s2 freq  -9612 path delay 9335
ptp4l[1879679.349]: master offset    -100 s2 freq  -9587 path delay 9335
ptp4l[1879680.249]: master offset     -73 s2 freq  -9590 path delay 9335
ptp4l[1879681.149]: master offset     -79 s2 freq  -9618 path delay 9344
ptp4l[1879682.049]: master offset     -76 s2 freq  -9639 path delay 9344
ptp4l[1879682.949]: master offset    -59 s2 freq  -9645 path delay 9329
ptp4l[1879683.849]: master offset    -31 s2 freq  -9634 path delay 9329
ptp4l[1879684.750]: master offset      22 s2 freq  -9591 path delay 9329
ptp4l[1879685.650]: master offset     -9 s2 freq  -9615 path delay 9337
ptp4l[1879686.550]: master offset    -31 s2 freq  -9640 path delay 9337
ptp4l[1879687.450]: master offset     -3 s2 freq  -9621 path delay 9337
ptp4l[1879688.350]: master offset    -15 s2 freq  -9634 path delay 9351
```

12. Problem Analysis

12.1 DMA Initialization Failed

If the GMAC driver's boot log displays: `DMA engine initialization failed`, it can be considered that there is an issue with the GMAC's working clock. First, you have to measure whether there is a clock on the clock pin, and whether the clock frequency and amplitude are normal. The main aspects to confirm are as follows:

- IOMUX error, check whether the clock pin register value is correct
- The clock direction and configuration do not match the hardware, refer to the clock settings in Chapter 4 of this document
- Check the clock tree and CRU registers to confirm the clock frequency and whether the clock is enabled

12.2 PHY Initialization Failure

If the GMAC driver's boot log displays messages such as "No PHY found" or "Cannot attach to PHY," it indicates that the PHY is not detected. The driver will first read the PHY's ID through the MDIO. It is possible to measure the MDC and MDIO waveforms to check if they are normal; this bus is similar to I2C, with a requirement that the MDC frequency be less than 2.5M. Generally, there are several reasons why the PHY might not be found:

- Verify if the MDC/MDIO IOMUX register values are correct.
- Check if the PHY power supply is normal.
- Incorrect Reset IO configuration.
- The Reset IO timing does not meet the PHY datasheet requirements; different PHYs have varying timing requirements. For specific configurations, refer to the DTS section of this document.
- Test if the MDIO/MDC waveforms are abnormal, with the MDC clock frequency required to be less than 2.5M.

12.3 Link Issues

If a Link issue occurs, an exclusion method can be applied by disconnecting the MDC/MDIO from the main controller and connecting it directly to a computer to check if the same issue persists on the computer side, which excluding software interference. This allows us to focus on investigating the impact of hardware. First, test whether TXN/P and RXN/P have Link waveforms.

If there is a continuous occurrence of Link up/Link down, possible reasons might be that the PHY has received incorrect data:

- In EEE mode, the waveform sent under incorrect delayline configuration may cause continuous link up/down.
- An incorrect clock supply to the PHY can also lead to this issue.

12.4 Data Inconsistency

First, check whether it is a TX issue, or an RX issue, or if both are problematic.

12.4.1 TX

Check if the TX packets of the eth0 interface are continuously increasing by using `ifconfig -a`. If it is 0, it may indicate that the network cable is not linked. By examining the node, you can determine if the network cable is connected; carrier 1 indicates link up, while 0 indicates link down. For example, RK3328:

```
console:/ # cat /sys/devices/platform/ff550000.ethernet/net/eth0/carrier
1
```

```
eth0      Link encap:Ethernet  HWaddr 16:21:8d:d9:67:0b  Driver rk_gmac-dwmac
          inet6 addr: fe80::c43d:3e5d:533:b7ea/64 Scope: Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:19 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 TX bytes:2848
          Interrupt:45
```

Assuming TX packets are continuously increasing, it indicates that TX data is being sent from the GMAC.

Connect the board card to the same local area network as the PC, and ping the PC from the board card while capturing packets on the PC end using a packet capturing tool (such as Wireshark). If data from the board card is captured, it indicates that TX data is passing through. If not, it is necessary to confirm where the TX anomaly occurred in the link and test the waveform of the GMAC's TX Clock and TX Data to rule out whether the MAC or PHY is the issue. The MAC can be checked in the following aspects:

- Verify the iomux of TX Clock/TX Data
- Whether the TXC clock is correct
- In RGMII mode, whether the Tx Delayline configuration is correct

The PHY side can also test the PHY's TXN/P signals to confirm whether the PHY is sending data; for different PHYs, their configurations may vary, and it is necessary to refer to their respective Datasheet.

12.4.2 RX

After the above troubleshooting confirms that it is not a TX issue, focus on troubleshooting RX; connect the network cable and check if the RX packets of the eth0 interface are continuously increasing through `ifconfig -a`. If it is 0, it indicates that GMAC RX has not received data.

```
eth0      Link encap:Ethernet  HWaddr 16:21:8d:d9:67:0b  Driver rk_gmac-dwmac
          inet6 addr: fe80::c43d:3e5d:533:b7ea/64 Scope: Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:341 errors:0 dropped:0 overruns:0 frame:0
          TX packets:26 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:48928 TX bytes:3741
          Interrupt:355
```

Similarly, you can test the PHY's RXN/P, as well as the GMAC's RX Clock/RX Data, to rule out whether it is a MAC or PHY issue. The MAC can be checked in the following aspects:

- Check the iomux of RX Clock/RX Data
- Whether the RXC clock is correct
- Whether the RGMII Tx Delayline configuration is correct
- Whether the Rx Delayline configuration is correct when in RGMII mode

Assuming TX packets are continuously increasing, but the Ethernet still does not communicate normally, it may be due to the following reasons:

- In RMII mode, the reference clock of MAC and PHY is not the same
- The PHY mode configuration is incorrect, for example, the hardware is configured in MII mode

12.5 TX queue0 timeout

It is believed that TX cannot transmit, which is generally caused by controller exceptions. These controller exceptions may be due to the following reasons:

- Clock issues: Check if the clock configuration is correct, refer to Chapter 3 of this document.
- PHY timing issues: Incorrect reset timing of PHY leads to incorrect clock provided by PHY.
- PHY hardware issues: Conflict detection occurs, preventing data transmission.
- Logic voltage is too low.